

El examen consiste en 10 preguntas. Las primeras 7 son teóricas, de corta respuesta y abarcan todo el temario de la materia. Las últimas 3 preguntas son prácticas y ameritan un breve desarrollo con mayor detalle.

Pregunta Teórica: ¿A qué se refiere el concepto de drift de un reloj? Explique algún algoritmo para mitigar su efecto.

El drift refiere al efecto de corrimiento del valor de un reloj respecto de otro de referencia. Es un efecto inevitable que produce la desincronización entre sistemas. Su efecto se puede mitigar mediante el Algoritmo de Cristian, donde el cliente envía un pedido del valor del reloj al servidor utilizado como referencia. Al recibir la respuesta, el cliente actualiza su reloj según $T_{server} + (T_{recepción} - T_{envío})/2$.

Pregunta Práctica: Utilizando una arquitectura Request-Reply, ejecute de forma remota la productoria de un arreglo de enteros de manera asíncrona. Mientras la ejecución transcurre, calcule la sumatoria. Por último, imprima ambos resultados. Utilice pseudocódigo o diagrama UML (secuencia, colaboración o actividades).

El patrón requesty-reply requiere de pares de mensajes por lo que es naturalmente un protocolo bloqueante. Sin embargo, se puede utilizar una técnica simple de encolar una acción para luego realizar pooling por su estado para realizar trabajos asíncronos:

```
# Client:

Main:
  Init connection('server')
  Init numeros: int[]

  req = RequestMsg('task_exec_productoria', numeros)
  connection.send_request(req)
  reply = connection.receive_reply()
  if reply.status == error:
    # fallo en la comunicación req-reply
    fatal_error()
  task_id = reply.body

  sumatoria = 0
  foreach n in numeros:
    sumatoria += n

  req = RequestMsg('task_get_result', task_id)
  reply = connection.receive_reply()
  if reply.status == error:
    # fallo en la comunicación req-reply
    fatal_error()
  productoria = reply.body

  print('Resultado: Sumatoria={sumatoria}, Productoria={productoria}')
  connection.close()
```

Server:

Main:

```
Init connection;
while connection.status != error
    req = connection.receive_request()
    if req.operation == 'task_exec_productoria':
        numeros = req.body
        task_id = exec_parallel_productoria(numeros)
        connection.send_reply(task_id)
    else if req.operation == 'task_get_result':
        task_id = req.body
        productoria = wait_parallel_result(task_id)
        connection.send_reply(productoria)
connection.close()
```